

C2000 BiSS-C Encoder Interface Software Guide

SDK v5.03.00

CONTENTS

1	Getting Started	1
1.1	Overview	1
1.2	Getting Started Steps	2
1.3	Minimum Requirements	2
1.4	Limitations	3
1.5	Installation and Directory Structure	3
2	Flowcharts	5
3	System Solution	9
3.1	Configuration	9
3.2	Build and Run the Example	13
3.3	System Test Structures and Arrays	17
3.4	Library Structures and Arrays	18
3.5	System-Level Functions	19
4	Encoder Interface Library	21
4.1	BiSS-C Library Integration	21
4.2	BiSS-C Library Porting to CPU2	26
4.3	Library Structures and Arrays	27
4.4	Encoder Interface API	30
4.5	Library Private Functions	33
4.6	Protocol Defines	36
5	Additional Resources	40
5.1	BiSS-C License	40
5.2	BiSS-C Protocol Description	40
5.3	Abbreviations and Acronyms	40
6	Changelog	42
6.1	C2000 MotorControl SDK Version 5.03.00.00	42
7	Important Notice	43
8	Disclaimer	45
	Index	46

GETTING STARTED

1.1 Overview

BiSS-C Encoder Interface by iC-Haus GmbH, is designed for serial transfer of digital data between linear, rotary or angle encoders, touch probes, accelerometers and the subsequent electronics, such as numerical controls, servo amplifiers and programmable logic controllers.

Note: BiSS is a trademark of iC-Haus GmbH. Developers must obtain their own BiSS license. Information can be found at biss-interface.com. For more details on the protocol and implementation aspects, see the [BiSS-C Interface](http://biss-interface.com) website.

The C2000 BiSS-C Encoder Interface Reference is divided into three parts:

1. Hardware and overall design. This is described in [C2000 BiSS-C Encoder Interface TI Design \(TIDM-1010\)](#).
2. Software: BiSS-C encoder interface Application Programming Interface (API)
3. Software: System solution demonstration

This document focuses on the API and system example.

1.1.1 Application Programming Interface (API)

The API provides functions and definitions used to implement the encoder interface.

Key Features of the library:

- Encoder interface integrated into a C2000 real-time microcontroller solution
- Meets the encoder interface clocking requirements
- Supports read and write of registers through the control data (CD) frame
- Cyclic Redundancy Check (CRC) calculations
- All source code is provided

1.1.2 System Example

Provides support functions to configure a C2000 Real-Time Control device to support BiSS-C.

Key Features:

- Provides a blueprint for how to integrate the library into a project.
- Processing to perform basic error checks, update multi-turn and single-turn position.
- Example to read and write to registers through control data.
- All source code is provided.

Note: For device-specific requirements, refer to the [TIDM-1010 BiSS-C Encoder Interface design guide](#).

1.2 Getting Started Steps

The recommended sequence to get started is:

1. Review and install the tools as described in *the minimum requirements*.
2. Run the *system example*.
3. If implementing your own system, refer to *BiSS-C library integration*.
4. Obtain your development license. BiSS is a trademark of iC-Haus GmbH. Developers must obtain their own BiSS license. Information can be found at biss-interface.com. For more details on the protocol and implementation aspects, see the [BiSS-C Interface website](#).

1.3 Minimum Requirements

The following development tools were used to build and test the release:

1.3.1 Development Environment

- Code Composer Studio v12.7.1
- C2000 Code Generation Tools v22.6.1.LTS
 - The code was built for abi = EABI
 - Optimization levels used for benchmarks: -o3 -mf 5
 - For more information, refer to the [C2000 Optimization Guide](#)

1.3.2 Software Development Kit

- C2000Ware Motor Control SDK v5_03_00_00
- C2000Ware for C2000 MCUs v5.03.00.00. C2000Ware is included within the Motor Control SDK.

1.3.3 Hardware

The hardware required is documented in [TIDM-1010](#). This includes:

- Supported C2000 LaunchPads
- C2000 device requirements
- The Position Manager BoosterPack ([BOOSTXL-POSMGR](#))
- A BiSS-C encoder with an appropriate shielded cable.

1.4 Limitations

Some features of BiSS-C have not been implemented in this communications demonstration. These features can be enabled by the system developer if required.

1. Topologies other than point-to-point.
2. Sequential (back-to-back) access to multiple registers. Single accesses are supported.
3. Control data command frames (CTS = 0).
4. The extension of the control-data frame by the slave. For example if the slave requires additional cycles to read or write to a register.
5. Broadcast commands
6. Some encoders require a higher current than the launchPad plus BOOSTXL_POSMGR can supply. If the encoder does not respond, or responds incorrectly, try providing power directly to the encoder from an external supply.

1.5 Installation and Directory Structure

1.5.1 Installing the Examples

After installing the tools described in [requirements](#), the library and example projects can be found in the following SDK directory:

1.5.2 Root Directory

The default install location is shown below:

Contents	Abbreviation	Default Location
Motor Control SDK	[MC_SDK_ROOT]	C:/ti/c2000/C2000Ware_MotorControl_SDK_<version>
System Example	[SYS_ROOT]	[MC_SDK_ROOT]/solutions/boostxl_posmgr/
API Library	[API_ROOT]	[MC_SDK_ROOT]/libraries/position_sensing/bissc

1.5.3 System Example Directory Structure

The system reference example source is located in the following directory structure with respect to [MC_SDK_ROOT]/solutions/boostxl_posmgr/:

System Reference Directory	Contents
[SYS_ROOT]/ccs/bissc/	Code Composer Studio projectspec file
[SYS_ROOT]/shared/include/bissc*.h	System Header file. Shared between devices
[SYS_ROOT]/shared/source/bissc*.c	System source code. Shared between devices
[SYS_ROOT]/shared/debug/bissc_vars.txt	CCS expressions import file.
[SYS_ROOT]/<device>/cmd	Linker command file. Device specific
[SYS_ROOT]/<device>/source/*bissc*.c	C28x C-source files. Device specific
[SYS_ROOT]/<device>/source/bissc*.syscfg	Device-specific pin-Mux and XBAR, CLB interconnect

1.5.4 Library Directory Structure

The API source is located in the following directory structure with respect to [MC_SDK_ROOT]/libraries/position_sensing/bissc:

API Library Reference	Contents
[API_ROOT]/ccs/<device>	Code Composer Studio projectspec file
[API_ROOT]/include	Header files
[API_ROOT]/source/pm_bissc_source.c	C28x communication C-source code
[API_ROOT]/source/pm_bissc_crc.c	C28x CRC C-source code
[API_ROOT]/source/pm_bissc<device>.syscfg	CLB Tool: tile configuration. Device specific.
[API_ROOT]/lib/pm_bissc_<device>.lib	Device-specific binary

FLOWCHARTS

An overview of the software architecture is shown in [Fig. 2.1](#).

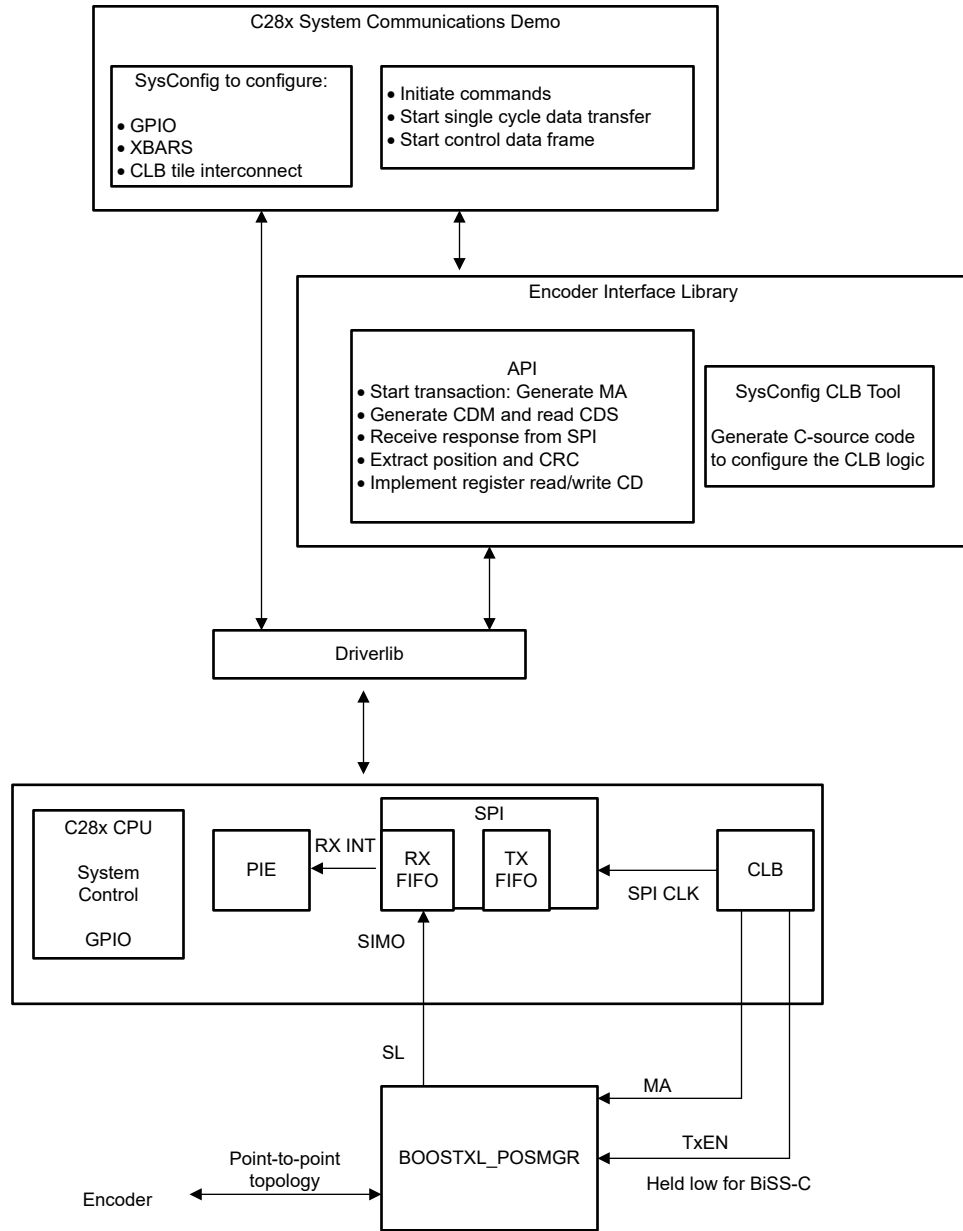


Fig. 2.1: Software Architecture.

Fig. 2.2 shows the software flow of the system example and the processing of control data.

Fig. 2.3 details the states of the control data state machine.

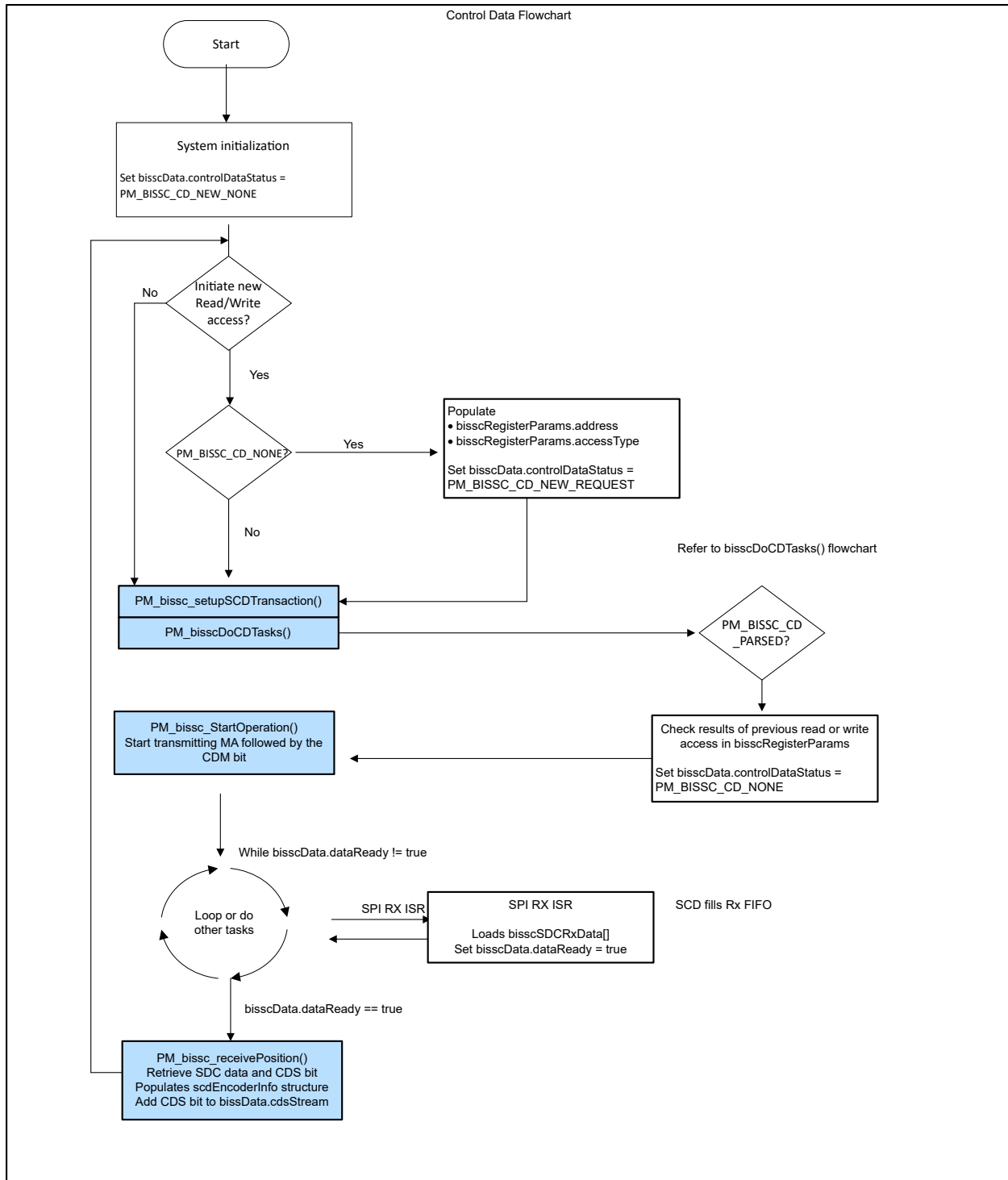


Fig. 2.2: Control Data Flowchart.

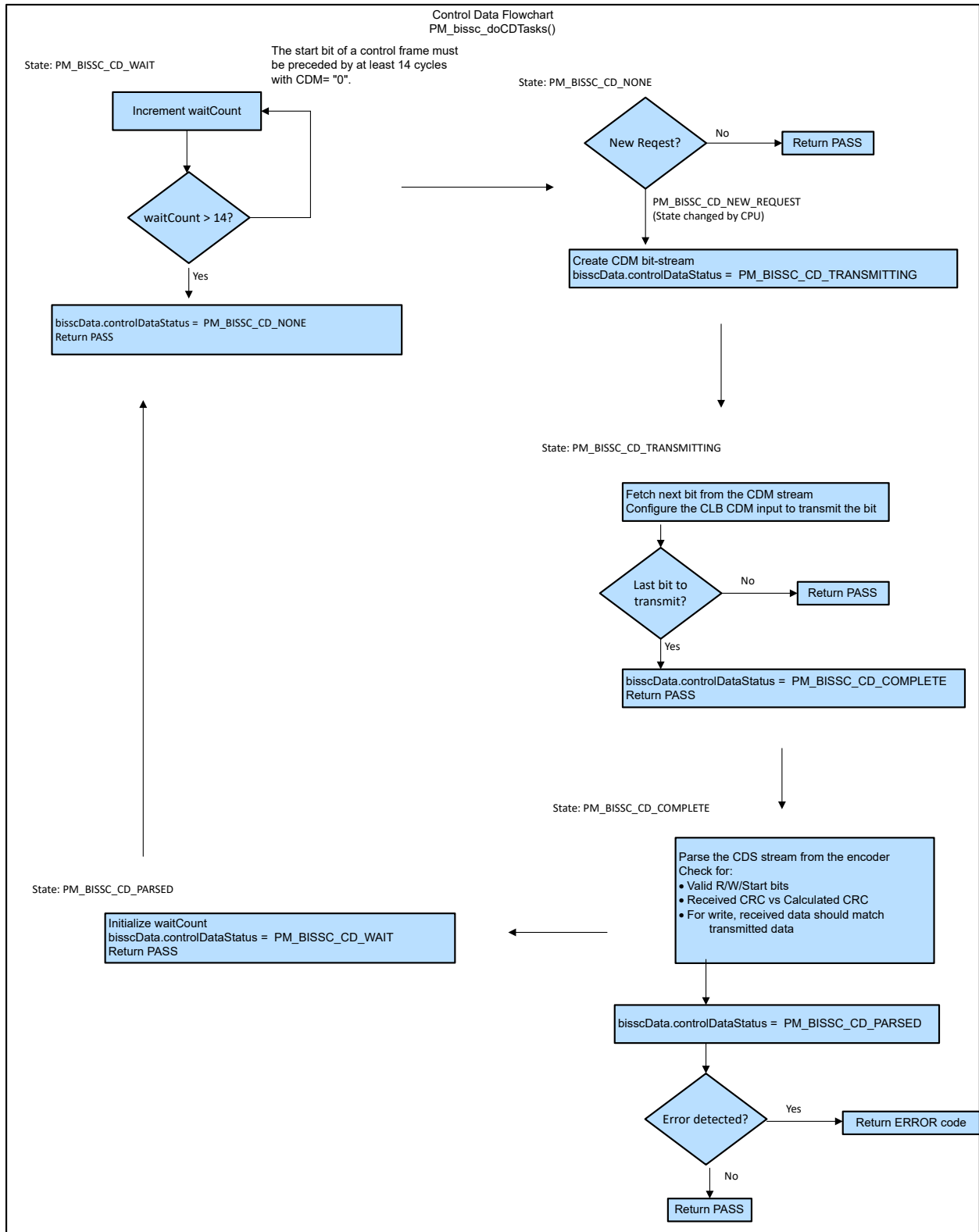


Fig. 2.3: Control Data State Machine.

SYSTEM SOLUTION

3.1 Configuration

3.1.1 System Example Configuration

If you wish to change the default configuration of the system example, edit the example's header file. The parameters that can be changed are shown below.

Listing 3.1: /solutions/boostxl_posmgr/shared/include/bissc.h

```
//  
// Number of multi-turn and single-turn bits.  
// In an application these may be read from the encoder/sensor itself. In this  
// demonstration they are hard-coded for simplicity.  
//  
//  
→ *****  
  
//  
// Lika EOH36-20-12  
//  
#define BISSC_SCD_MT_BITS    12U  
#define BISSC_SCD_ST_BITS    20U  
//  
  
//  
// icHaus MHM Evaluation board  
//  
//#define BISSC_SCD_MT_BITS    16U  
//#define BISSC_SCD_ST_BITS    12U  
  
//  
→ *****  
//  
// Specify the BiSS-C clock rate (MA) in MHz  
//  
//  
→ *****
```

(continues on next page)

(continued from previous page)

```

#define BISSC_MA_CLOCK 8.0f // MHz

//
// *****
//
// For register read/write testing, the address of the bank select register
// is used. This is standard on encoders and safe to read/write from.
//
// The test keeps track of BISSC_READ_COUNT reads in the register_read_data
// array.
//
// *****
#define BISSC_READ_COUNT          8U
#define BISSC_BANK_SELECT         0x40U
#define BISSC_CD_BUSY             1U
#define BISSC_CD_SETUP_COMPLETE  0U
extern uint16_t bisscRegisterReadData[BISSC_READ_COUNT];

//
// *****
//
// CRC related declarations
//
// The leading 1 on the polynomial (poly) is implied by the size
// The most common polynomials are:
//
// Control Data (CD):
//       $x^4 + x + 1 \rightarrow 10011 \rightarrow 0x13$ 
//      leading 1 is implied use: Poly = 3, poly bits = 4
//
// Single Cycle Data (SCD):
//       $x^6 + x + 1 \rightarrow 1000011 \rightarrow 0x43$ 
//      leading 1 is implied use: Poly = 3, poly bits = 6
//
// Specify the CRC starting value. For BiSS-C this is typically 0.
//
//
// Other BiSS polynomials:
//
//  $x^5 + x^2 + X^0 = 0x25$ 
//      ---> Poly = 5, poly bits = 5
//
//  $x^{16} + x^{15} + x^{12} + x^7 + X^6 + x^4 + x^3 + X^0 = 0x190D9$ 
//      ---> Poly = 0x90D9, poly bits = 16
//
// *****
#define BISSC_CD_CRC_POLY_BITS    4U
#define BISSC_CD_CRC_POLY        0x03U

```

(continues on next page)

(continued from previous page)

```

#define BISSC_CD_CRC_START      0U
#define BISSC_CD_CRCTABLE_SIZE  256U

#define BISSC_SCD_CRC_POLY_BITS  6U
#define BISSC_SCD_CRC_POLY      0x03U
#define BISSC_SCD_CRC_START      0U
#define BISSC_SCD_CRCTABLE_SIZE  256U

// #define BISSC_SCD_CRC_POLY_BITS  16U
// #define BISSC_SCD_CRC_POLY      0x90D9U
// #define BISSC_SCD_CRC_START      0U
// #define BISSC_SCD_CRCTABLE_SIZE  256U

//
// *****
//
// Delay time, in micro-seconds, between each transaction
// This time does not include the parse of the SCD and CDS.
// In a typical application this time would be controlled by a timer_
// interrupt.
// In this demonstration, a simple while() loop with a delay is used.
//
// The value chosen was based on the read access time of one of the
// tested encoders. This allows the reads to occur even with the limitations
// of the CD state machine implementation. If the state machine is updated
// to allow for the encoder to extend the read, or write, cycle this time
// can be reduced. Other encoders may not require this much time. Refer
// to the encoder specifications.
//
//
// *****
#define BISSC_IDLE_DELAY_US      200u1

//
// *****
//
// Power-on to normal operation delay 1.5 seconds. Refer to the specifications
// for a specific encoder.
//
//
// *****
#define BISSC_POWER_ON_DELAY_US 1500000u1

//

```

3.1.2 Library Configuration

If you wish to change the default configuration of the library, edit the library's public header file. The parameters that can be changed are shown below.

Note:

- If you change the library configuration then recompile **both** the library **and** the system example.

Listing 3.2: /libraries/position_sensing/bissc/include/pm_bissc_include.h

```
//
// #define BISSC_ENCODER_HAS_CD_INTERFACE
//     Some encoders do not support the control-data CD interface.
//     0: the code associated with the control-data frames is not executed.
//     else: the code associated with control-data frames is executed.
//
// #define PM_BISSC_SPI    SPIA_BASE, SPIB_BASE, SPIC_BASE, SPID_BASE
//     Define which SPI peripheral is used. This MUST match the instance
//     the CLB is clocking and be connected to the RS485 interface logic.
//
//     Note: The CLB output configuration and SPI configuration may need
//     to be modified if this selection is changed. Refer to the
//     design guide for more information.
//
// #define PM_SPI_FIFO_MAX_LEVEL
//     This is the depth of the SPI FIFO. If you know the message returned
//     from the encoder will always use fewer levels it can be lowered.
//     This parameter is only used when reading the message out of the
//     SPI FIFO and can save cycles.
//
// #define PM_BISSC_SPI_FIFO_WIDTH
//     Word size used by the SPI peripheral. You shouldn't have to change
//     this definition.
//
// #define PM_BISSC_ENCODER_HAS_CD_INTERFACE    1U
// #define PM_BISSC_SPI_FIFO_WIDTH             12U
// #define PM_BISSC_SPI_FIFO_MAX_LEVEL         16U
//
// SPI selection specified here and the CLB output
// configuration, and the system control setup in the
// communications demo must match. If one is changed,
// the other must be changed as well.
//
// #if defined(_F28P65X) || defined(F28P65x_DEVICE)
//     #define PM_BISSC_SPI                SPID_BASE
//     #define PM_BISSC_INT_SPI_RX         INT_SPID_RX
//     #define PM_BISSC_CLK_GEN_BASE       CLB3_BASE
//     #define PM_BISSC_FSM_BASE           CLB4_BASE
// #else
//     #error Device should match a predefined symbol in the build options.
```

(continues on next page)

(continued from previous page)

#endif

//

3.2 Build and Run the Example

This example provides a roadmap for integration of the library into an application.

Note: For Code Composer Studio information, refer to the [Code Composer Studio User's Guide](#).

3.2.1 Hardware Connections

Setup the hardware as described in the [TIDM-1010](#) design guide.

3.2.2 Import and Run Example Project

The system example directory, [MC_SDK_ROOT]/solutions/boostxl_posmgr/<device>, contains a Code Composer Studio projectspec file. To import the project into CCS, follow these steps:

1. Install the minimum required tools. Refer to [Minimum Requirements](#).
2. Import the example system demonstration.
 1. In Code Composer Studio (CCS), click *Project -> import CCS Projects..*
 2. Navigate to the system example directory: [MC_SDK_ROOT]/solutions/boostxl_posmgr/<device>
 3. In the ccs/bissc/ directory, select the bissc_systemtest.projectspect file
 4. Click *Finish*.
 5. In the CCS Project Explorer you should now see the demo and library projects as shown in [Fig. 3.1](#).

Note: When you import the system example, CCS will also import the associated library project.

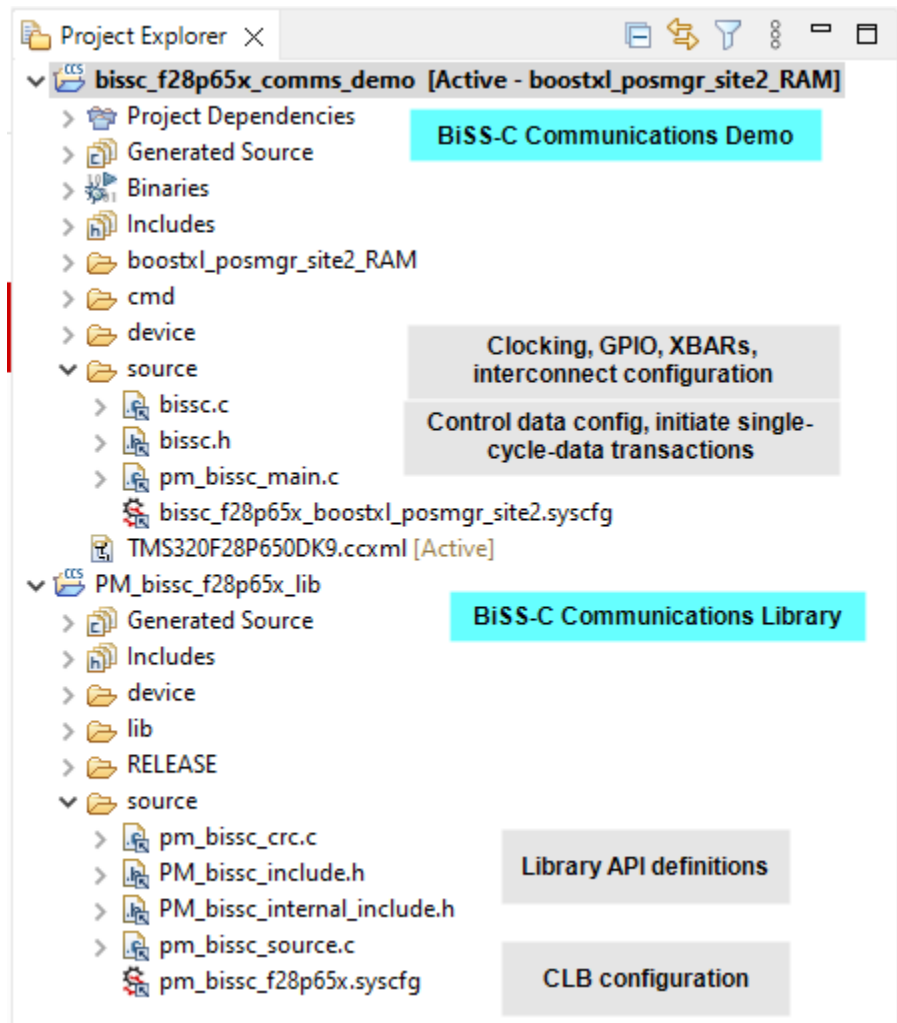


Fig. 3.1: Code Composer Studio Project Explorer Window.

3. Select the build configuration of choice
 1. To select the build configuration: *right-click on the project*
 2. Select: *build configurations* and *set active*.
 3. Select the build configuration.

The build configurations are:

boostxl_posmgr_site2_RAM: Builds the solution to load and run from RAM. The pin-out corresponds to the LaunchPad + BOOSTXL_POSMGR site2 as described in [TIDM-1010](#) design guide.

boostxl_posmgr_site2_FLASH: The solution loads the demo to flash. At run-time, critical portions are copied to execute from RAM.

1. Select the active board configuration

Note: This step may or may not apply to your project. The image shown below is provided as an

example only.

1. Some projects may include two board configuration files (.ccxml) as shown in Fig. 3.2. This occurs when the LaunchPad and ControlCard use different JTAG connections.
2. *Right-click* on the board configuration matching your hardware.
3. Select *Set as active target configuration*

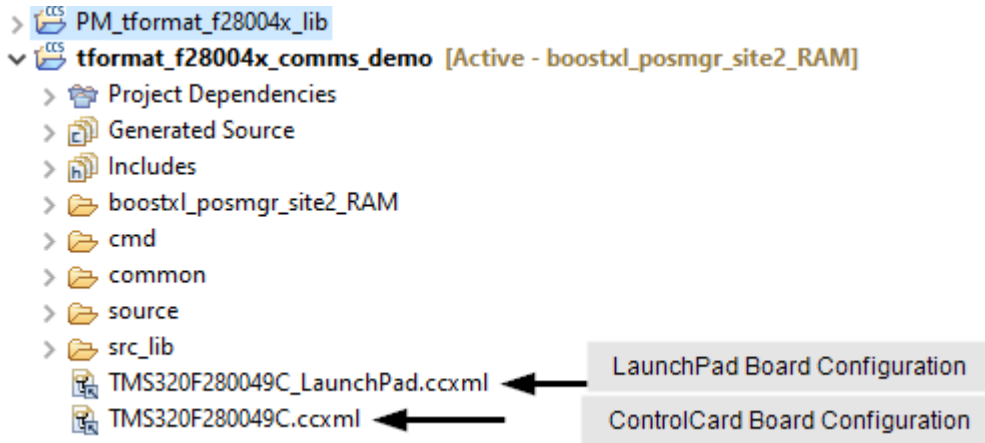


Fig. 3.2: Board Configurations

1. Build the system example project
 1. In the project manager window, *Right-click* on the project name `bissc_<device>_boostxl_comms_demo`
 2. Select *Rebuild Project*
 3. Observe the console window for any errors or for successful completion of the build.
 4. Once the build completes, without errors, execute the project by selecting *Run* → *Debug*
2. Populate the expressions window
 1. *Right-click* on the expressions window
 2. Select *import*
 3. Browse to the location `[SYS_ROOT]/shared/debug/`
 4. Select the file `bissc_vars.txt` and click *open*
 5. At the top of the expressions window, enable *continuous refresh* as shown in Fig. 3.5.

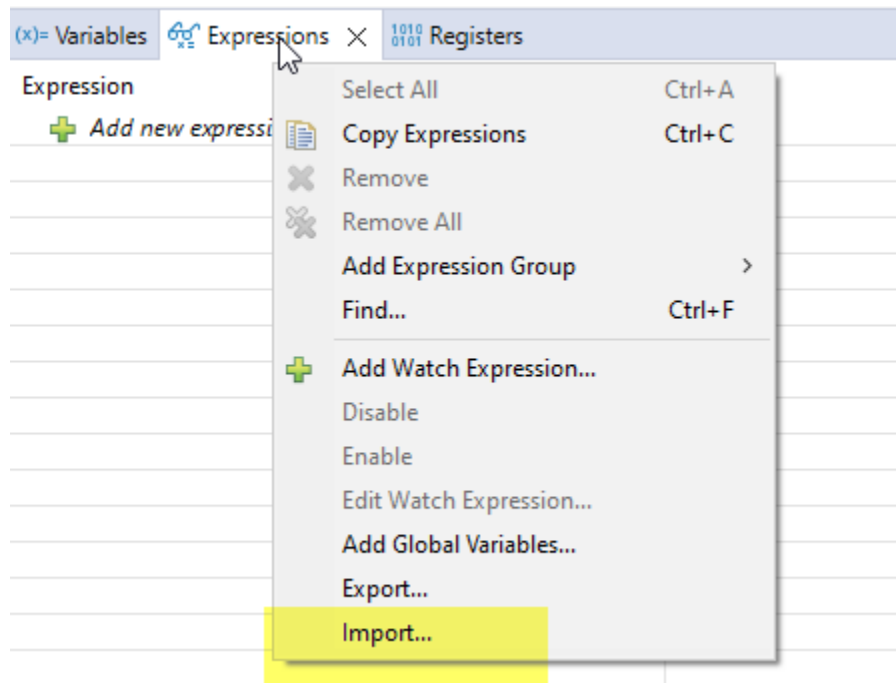


Fig. 3.3: Code Composer Studio Import Expressions.

Expression	Value
> bisscRegisterReadData	[1,3,5,1,3...]
> cdParams	{cdStatus=PM_BISS_CD_TRANSMITTING,cdmStream...
> scdRxData	[65535,65444,17529,38015,3026...]
> scdParams	{dataReady=0 '\x00',crcBits=6,crcPoly=3,crcStart=0,cr...
▼ bisscDiagnostics	{scdCRCFailCount=0,scdPassCount=54871,cdPassCou...
(x)= scdCRCFailCount	0
(x)= scdPassCount	54871
(x)= cdPassCount	54872
(x)= cdFailCount	0
(x)= cdRWInvalidCount	0
(x)= cdWUnsupportedCount	0
(x)= cdWriteFailCount	0
(x)= cdCRCFailCount	0
▼ encoderInfo	{positionMT=65534,positionST=2321,position=2684295...
(x)= positionMT	0x0000FFFE (Hex)
(x)= positionST	0x00000911 (Hex)
(x)= position	0x00000000FFFE911 (Hex)
(x)= error	0x0001 (Hex)
(x)= warning	0x0001 (Hex)
+ Add new expression	

Fig. 3.4: Code Composer Studio Expressions Window.

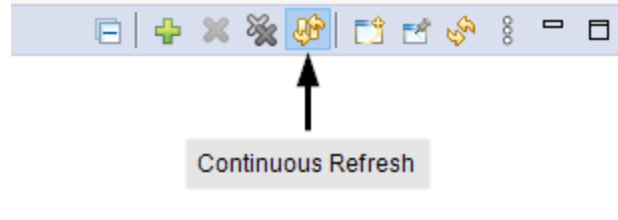


Fig. 3.5: Enable Continuous Refresh.

3. Run the code

1. Press the *Run/Resume* button
2. Manually turn the shaft of the motor or encoder
3. Monitor the signals and variables in the expressions window. These structures and arrays are described in the *globals* section.

3.3 System Test Structures and Arrays

The following variables are used by the system only for testing. Monitor these in the expressions window as shown in the *System Example*.

Listing 3.3: System test structure and array:
/boostxl_posmgr/f28p65x/source/pm_bissc_main.c

```
//
//
// *****
//
// A circular buffer for register reads. Each read is performed after a write
// access. As provided the reads should alternate between 1, 3, and 5.
//
//
// *****
uint16_t bisscRegisterReadData[BISSC_READ_COUNT];
//
// *****
//
// Test structure to log pass/fail counts returned by the API
//
//
// *****
bissc_diagnosticsStruct bisscDiagnostics;
bissc_diagnosticsStruct_Handle bisscDiagnostics_handle = &bisscDiagnostics;
//
```

3.4 Library Structures and Arrays

The following structures and arrays interact with the library. They can be monitored in the expressions window as described in *the System Example*.

Listing 3.4: Global structures and arrays:
/boostxl_posmgr/f28p65x/source/pm_bissc_main.c

```
//
//
// *****
// Array and structures required to interface with the library
//
// *****
volatile uint32_t scdRxData[BISSC_SPI_MAX_FIFO_LEVEL];

PM_bissc_cdStruct cdParams;
PM_bissc_cdStruct_Handle cdParams_handle = &cdParams;

PM_bissc_scdStruct scdParams;
PM_bissc_scdStruct_Handle scdParams_handle = &scdParams;

PM_bissc_registerStruct registerParams;
PM_bissc_registerStruct_Handle registerParams_handle = &registerParams;

PM_bissc_encoderStruct encoderInfo;
PM_bissc_encoderStruct_Handle encoderInfo_handle = &encoderInfo;

uint16_t bisscCRtableSCD[PM_BISSC_SCD_CRCTABLE_SIZE];

#ifdef(PM_BISSC_ENCODER_HAS_CD_INTERFACE)
uint16_t bisscCRtableCD[PM_BISSC_CD_CRCTABLE_SIZE];
#endif
//
```

The definitions of these can be found in *the Library Variables section*.

3.5 System-Level Functions

This section describes functions which act at the system-level. This includes initialization of the C2000 device, start-up functions and interrupt service routine.

Many of these functions make calls to the BiSS-C Library which is described in *API Library Documentation*.

3.5.1 System Initialization Functions

Most of the initialization function calls to driverlib or to functions generated by the system .syscfg file. The `bissc_init()` function leverages these plus performs some additional BiSS specific initializations.

void **bissc_init** (PM_bissc_scdStruct_Handle, PM_bissc_cdStruct_Handle)

Initialize the BiSS-C Interface.

This example function initializes variables, the CLB and other peripherals required to integrate the BiSS-C encoder interface communications library into a C28x application.

This includes:

- Configure the device GPIO, XBAR, SPI. This code is generated by the demo project's syscfg file
- Load the CLB tile configuration from the library
- Configure the CLB to generate the required duty cycle for the BiSS-C MA signal (clock)
- Initialize SCD and CD parameters used by the library
- Configure the SPI interrupt
- Enable power to the encoder

Returns None.

3.5.2 System Run-time Functions

The following system-level functions are called during run-time.

void **bissc_runSystem** (PM_bissc_scdStruct_Handle, PM_bissc_cdStruct_Handle,
PM_bissc_registerStruct_Handle, PM_bissc_encoderStruct_Handle,
bissc_diagnosticsStruct_Handle) __attribute__((ramfunc)) __attribute__((noreturn))

Run the communication demonstration.

This example function demonstrates how to call into the BiSS-C library to perform single-cycle-data transactions and read/write from registers.

This function runs an infinite while() loop:

- a. Execute the command data state-machine and check for pass/fail status
- b. Initiate single-cycle data transactions
- c. Wait for the SCD response be received and read in the SPI RX ISR routine
- d. Call the library's parser function to extract the position, CRC and CDS bit from the received SCD stream
- e. Check the parser's return value for pass or fail status

Returns None.

void **bissc_setupCDTest** (PM_bissc_cdStruct_Handle cdParams, PM_bissc_registerStruct_Handle registerParams) __attribute__((ramfunc))

Test control data, CD, register read and write.

This function configures a CDM stream for register accesses.

The BISSC_BANK_SELECT register (address 0x40) is used for both read and write accesses. This register is typically safe to write to without causing any undesired issues.

The test routine consists of 6 tests numbered 0-5.

Test 0, 2, and 4:

- Setup a CDM stream to read the BISSC_BANK_SELECT register

Test 1, 3, and 5:

- Log the previous read in the bisscRegisterReadData array
- Setup a CDM stream to write to the BISSC_BANK_SELECT register.

Note: As provided:

- The bisscRegisterReadData[] is a circular buffer of size BISSC_READ_COUNT. Old values will be overwritten.
 - The value written to the register corresponds to the test number (1, 3, or 5)
 - This test assumes the value written is a valid bank for the connected encoder.
 - The first read will depend on the value in the register at startup. The subsequent reads should alternate between the written values (1, 3, 5, 1, 3, 5...)
-

Parameters

- **cdParams** – Control data frame parameters
- **registerParams** – Register address, access type and data

Returns None

void **bissc_spiRxISR** (void) __attribute__((interrupt("HPI"))) __attribute__((ramfunc))

Service the SPI FIFO receive interrupt.

This interrupt service routine copies the contents of the SPI RX FIFO into the scdRxData array

Note: This interrupt service routine is defined as a high-priority interrupt for performance reasons. This means the ISR can not be interrupted by another ISR (no nesting allowed). Refer to the [C2000 compiler guide](#) for more information.

ENCODER INTERFACE LIBRARY

4.1 BiSS-C Library Integration

Reference the included System Example as a blueprint for integrating the library into an application. This section provides an overview of the steps required.

4.1.1 Resolve Resource Conflicts

1. Review the resources used by the Encoder Interface. The resource utilization is described in [TIDM-1010](#).
2. Resolve any resource conflicts with your specific application. Include the CLB tiles, XBARS and GPIO pins in this evaluation. In some cases, a specific resource was used due to the LaunchPad pinout. Your own application may have different hardware requirements and limitations. For example, a specific CLB tile may be able to access a GPIO through a specific XBAR.
3. Implement and test changes using the stand-alone example. The booster pack can be connected to a LaunchPad or ControlCard through wires to test the changes.
4. Proceed with integration into your application.

SPI Instance

If you change the SPI instance make it matches in the following locations:

1. System communication demo .syscfg file
2. Library pm_bissc_include.h file

4.1.2 Add Header Files

Include the following header files in the project source:

1. BiSS-C library header file: `#include "pm_bissc_include.h"`
2. System functions header file: `#include "bissc.h"`

4.1.3 Update Code Composer Studio Search Locations

CCS searches specific locations for header files and the library (.lib) files. Referring to the *directory structure* and update the search locations by the following steps:

1. In Code Composer Studio Project Explorer window: *Right click project name*
2. Select: *Properties -> Build -> C2000 Compiler Options -> Include Options*
3. Add the **directory path for each include file** to the compiler search path.
4. Once again, *Right click project name*
5. Select: *Properties -> Build -> C2000 Linker -> File search Path.*
6. Include the library binary in the linker options. For example: `pm_bissc_f28p65x.lib`
7. Add the directory path for the lib to the library search path.

4.1.4 Add System-Level Functions

The System Example includes functions which you may wish to include in your project. Depending on your specific needs, you can copy functions or include the source files from the example by following these steps:

1. In Code Composer Studio: *Right click project name*
2. Select: *Add Files*
3. Select the file(s) to include.

4.1.5 Create Global Structures

These structures are documented in the *the System Example*.

The library requires global variables to share data between modules and with the system.

Listing 4.1: Global structures and arrays
/boostxl_posmgr/f28p65x/source/pm_bissc_main.c

```
//
//
→ *****
//
// Array and structures required to interface with the library
//
//
→ *****
volatile uint32_t scdRxData[BISSC_SPI_MAX_FIFO_LEVEL];

PM_bissc_cdStruct cdParams;
PM_bissc_cdStruct_Handle cdParams_handle = &cdParams;

PM_bissc_scdStruct scdParams;
PM_bissc_scdStruct_Handle scdParams_handle = &scdParams;

PM_bissc_registerStruct registerParams;
PM_bissc_registerStruct_Handle registerParams_handle = &registerParams;
```

(continues on next page)

(continued from previous page)

```

PM_bissc_encoderStruct encoderInfo;
PM_bissc_encoderStruct_Handle encoderInfo_handle = &encoderInfo;

uint16_t bisscCRCTableSCD[PM_BISSC_SCD_CRCTABLE_SIZE];

#ifdef(PM_BISSC_ENCODER_HAS_CD_INTERFACE)
uint16_t bisscCRCTableCD[PM_BISSC_CD_CRCTABLE_SIZE];
#endif

//

```

There are additional globals that are used by this specific system example. These are not required for BiSS-C communication, but may be useful in your application. These are shown below.

Listing 4.2: /boostxl_posmgr/f28p65x/source/pm_bissc_main.c

```

//
//
→ *****
//
// A circular buffer for register reads. Each read is performed after a write
// access. As provided the reads should alternate between 1, 3, and 5.
//
//
→ *****
uint16_t bisscRegisterReadData[BISSC_READ_COUNT];

//
→ *****
//
// Test structure to log pass/fail counts returned by the API
//
//
→ *****
bissc_diagnosticsStruct bisscDiagnostics;
bissc_diagnosticsStruct_Handle bisscDiagnostics_handle = &bisscDiagnostics;

//

```

4.1.6 Initialize the System

Refer to the example *system initialization functions*. Some functions may require modification for your specific system. Check that the `BISS_FREQ_DIVIDER` is correctly defined for your device CLB clock frequency. This is defined in the `system_bissc.h` header file.

4.1.7 Implement Background Loop, SPI ISR

Use the provided *system run-time functions* as a template, implement the following functions:

End of single-cycle data interrupt (SPI RX ISR). Populate the RX data array with the content of the SPI RX FIFO.

Listing 4.3: SPI ISR: `/boostxl_posmgr/shared/source/bissc.c`

```
//
extern PM_bissc_scdStruct scdParams;
interrupt void bissc_spiRxISR(void)
{
    uint16_t i;
    for (i = 0; i < PM_BISSC_SPI_FIFO_MAX_LEVEL; i++)
    {
        scdRxData[i] = SPI_readDataNonBlocking(PM_BISSC_SPI)
            & scdParams.spiFIFOMask;
    }
    SPI_clearInterruptStatus(PM_BISSC_SPI, SPI_INT_RXFF_OVERFLOW);
    SPI_clearInterruptStatus(PM_BISSC_SPI, SPI_INT_RXFF);
    scdParams.dataReady = true;
    Interrupt_clearACKGroup(INTERRUPT_ACK_GROUP6);
}
//
```

4.1.8 Send MA and Receive Response

An example of the run-time flow is shown below. The library functions called are documented in the *Library Public API section*.

Listing 4.4: Run-time flow example: `/solutions/boostxl_posmgr/shared/source/bissc.c`

```
//
uint16_t status;
while(1)
{
    //
    // Setup a single-cycle data transaction
    // Check if a command data frame has been parsed or can be initiated
    // Run the command data state-machine
    // Check for errors from the command-data state-machine
    //
    PM_bissc_setupSCDTransaction(scdParams);
    #if PM_BISSC_ENCODER_HAS_CD_INTERFACE
    bissc_setupCDTest(cdParams, registerParams);
    #endif
}
```

(continues on next page)

(continued from previous page)

```

PM_bissc_doCDTasks(cdParams, registerParams);
if(cdParams->cdState == PM_BISSC_CD_PARSED)
{
    enum cdStatus cdStatus = cdParams->cdStatus;
    switch(cdStatus)
    {
        case PM_BISSC_CD_READ_PASS:
            bisscDiagnostics->cdReadPassCount++;
            break;

        case PM_BISSC_CD_WRITE_PASS:
            bisscDiagnostics->cdWritePassCount++;
            break;

        case PM_BISSC_CD_WRITE_FAIL:
            bisscDiagnostics->cdWriteFailCount++;
            break;

        case PM_BISSC_CD_CRC_FAIL:
            bisscDiagnostics->cdCRCFailCount++;
            break;

        case PM_BISSC_CD_READ_WRITE_INVALID:
            bisscDiagnostics->cdRWInvalidCount++;
            break;

        case PM_BISSC_CD_READ_WRITE_UNSUPPORTED:
            bisscDiagnostics->cdRWUnsupportedCount++;
            break;

        default:
            bisscDiagnostics->cdFailCount++;
            break;
    }
}
#endif // end PM_BISSC_ENCODER_HAS_CD_INTERFACE

//
// Clearing the single-cycle data array is not required.
// This is done to aid in debug.
//
memset((void *)scdRxData, 0, sizeof(scdRxData));

//
// Start the single-cycle data frame MA clock
// Wait for the SPI to read the data
// Parse the position, error and warning
// Check for errors
//
PM_bissc_startOperation(scdParams, BISSC_FREQ_DIVIDER);
while(scdParams->dataReady != true) {}

```

(continues on next page)

(continued from previous page)

```

        status = PM_bissc_receivePosition(scdParams, cdParams, encoderInfo);
        if(status != PM_BISSC_PASS)
        {
            bisscDiagnostics->scdCRCFailCount++;
            ESTOP0; // Halt PC on failure
        }
        else
        {
            bisscDiagnostics->scdPassCount++;
        }
        DEVICE_DELAY_US(BISSC_IDLE_DELAY_US);
    }
    //

```

4.2 BiSS-C Library Porting to CPU2

Some devices include a C28x CPU2. This section provides an overview of the steps to run the encoder interface from CPU2 instead of CPU1.

4.2.1 Resolve Resource Conflicts

If you are incorporating the library into your own project on CPU2, it is suggested to get a simple example running on CPU1 with any and all resource conflicts within your application resolved. Then focus on porting from CPU1 to CPU2. For information on incorporating the library into your own application, refer to *BiSS-C Library Integration*.

4.2.2 CPU1 Project

The project must be built with the pre-defined symbol CPU1.

The CPU1 sub-system initialization will include these steps:

- Initialize the device
- Allocate desired global memory blocks to CPU2
- Configure the CLB tiles for BiSS-C
- Allocate the peripheral access control and CPU for CLB tiles to CPU2
- Configure the PinMux, CLB_INPUT/OUTPUTXBAR, and CLB MUX as needed.
- Allocate needed GPIO pins to CPU2.
- Allocate the SPI peripheral to CPU2
- Boot CPU2
- Continue configuration of CPU1 sub-system

For F2837xD:

- The CLBx Tile is clocked by the ePWMx clock. That is, CLB1 is clocked by ePWM1 clock CLB2 by ePWM2, and so forth. This means the ePWMx module must also be assigned to CPU2 as well.

4.2.3 CPU2 Project

The project must be built with the pre-defined symbol CPU2.

The CPU2 sub-system initialization will include these steps:

- Enable the clock to the CLB tiles
- Set the BiSS-C frequency for MA (master clock)
- Setup the SPI interrupt in the PIE vector table, PIE IER register, and CPU IER register
- Enable the SPI peripheral

For F2837xD:

- To enable the clock to the CLB Tiles, enable the clocks to the corresponding ePWMx modules. For example: enable ePWM1 clock CPU2 will also enable the clock to the CLB1 Tile.

4.3 Library Structures and Arrays

The library uses the following structures and arrays.

Listing 4.5: Single-cycle-data processing parameters: /libraries/position_sensing/bissc/pm_bissc_include.h

```
//
→ *****
//
// Parameters needed to process single-cycle-data
//
//
→ *****
typedef struct PM_bissc_scdStruct {
    volatile bool    dataReady;    // false: not ready. true: ready
    uint16_t         crcBits;      // Size of polynomial
    uint16_t         crcPoly;      // Polynomial
    uint16_t         crcStart;     // CRC initial value
    uint16_t         crc_M;        // Mask value to extract the CRC
    uint16_t         spiFIFOWidth; // SPI FIFO word size
    uint16_t         spiFIFOMask;  // Mask for a SPI FIFO word
    uint16_t         spiFIFOLevel; // SPI FIFO depth
    uint16_t         numDataClocks; // Number of clocks required to receive SCD
    uint16_t         positionSTBits; // Number of single-turn bits
    uint16_t         positionST_S;  // Shift to extract ST bits
    uint16_t         positionMTBits; // Number of multi-turn bits
    uint16_t         positionMT_S;  // Shift to extract MT bits
    uint16_t         positionBits;  // Total position bits (MT + ST)
    uint32_t         positionMT_M;  // Mask to extract MT bits
    uint32_t         positionST_M;  // Mask to extract ST bits
    uint64_t         position_M;    // Mask to extract the total position
    uint64_t         posErrWarnBits; // Number of error and warn bits
} PM_bissc_scdStruct;

typedef PM_bissc_scdStruct * PM_bissc_scdStruct_Handle;
```

(continues on next page)

(continued from previous page)

//

Listing 4.6: Control-data frame processing: /libraries/position_sensing/bissc/pm_bissc_include.h

```
//
→ *****
//
// Control-data frame parameters and states
//
//
→ *****
enum cdState // Control data state machine
{
    PM_BISSC_CD_NEW_REQUEST, // New read or write request pending
    PM_BISSC_CD_TRANSMITTING, // Read or write is in progress
    PM_BISSC_CD_COMPLETE, // Read or write has completed
    PM_BISSC_CD_PARSED, // CD data has been parsed
    PM_BISSC_CD_WAIT, // BiSS required wait between CD frames
    PM_BISSC_CD_NONE
};

//
// The cdStatus will remain until either the application clears it (sets it to
// PM_BISSC_CD_PASS) or a new CD Request is made.
//
// PASS No error detected
// WRITE_FAIL Write access failure. The data echoback (received
// data) did not match the data transmitted
// CRC_FAIL: Calculated CRC did not match received CRC
// READ_WRITE_INVALID The combination of R+W is invalid per the
// protocol specification
// READ_WRITE_UNSUPPORTED Unsupported feature. Likely the encoder requires
// wait-states to access the register
//
enum cdStatus
{
    PM_BISSC_CD_PASS,
    PM_BISSC_CD_WRITE_FAIL,
    PM_BISSC_CD_WRITE_PASS,
    PM_BISSC_CD_READ_FAIL,
    PM_BISSC_CD_READ_PASS,
    PM_BISSC_CD_CRC_FAIL,
    PM_BISSC_CD_READ_WRITE_INVALID,
    PM_BISSC_CD_READ_WRITE_UNSUPPORTED
};

typedef struct PM_bissc_cdStruct{
    enum cdState cdState; // State machine current state
    enum cdStatus cdStatus; // Pass or Fail status
```

(continues on next page)

(continued from previous page)

```

    uint32_t   cdmStream;           // Least significant bit is next CM to transmit
    uint32_t   cdsStream;           // CDS stream without the CRC
    uint16_t   crcBits;              // Size of CRC polynomial
    uint16_t   crcPoly;              // CRC polynomial without the leading 1
    uint16_t   crcStart;             // CRC initial value
} PM_bissc_cdStruct;

typedef PM_bissc_cdStruct * PM_bissc_cdStruct_Handle;

//

```

Listing 4.7: Register access processing: /libraries/position_sensing/bissc/pm_bissc_include.h

```

//
→ *****
//
// Register access parameters used to specify address and access type
//
//
→ *****
enum cdRegisterAccessType
{
    PM_BISSC_REGISTER_READ = 2U,    // R:W = 1 0 = read access
    PM_BISSC_REGISTER_WRITE = 1U    // R:W = 0 1 = write access
};

typedef struct PM_bissc_registerStruct
{
    uint16_t   address;              // Address of register to access
    uint16_t   rxData;               // Data read from the register
    uint16_t   txData;               // Data to write to the register
    enum cdRegisterAccessType   accessType; // Specify read or write access
} PM_bissc_registerStruct;

typedef PM_bissc_registerStruct * PM_bissc_registerStruct_Handle;

//

```

Listing 4.8: Encoder information: /libraries/position_sensing/bissc/pm_bissc_include.h

```

//
→ *****
//
// Encoder information parsed from the single-cycle-data.
//
//
→ *****
typedef struct PM_bissc_encoderStruct
{
    uint32_t   positionMT;           // Multi-turn position

```

(continues on next page)

(continued from previous page)

```

uint32_t positionST;    // Single-turn position
uint64_t position;     // MT::ST position
uint16_t error;        // Error bit
uint16_t warning;      // Warning bit
} PM_bissc_encoderStruct;

typedef PM_bissc_encoderStruct * PM_bissc_encoderStruct_Handle;

//

```

Listing 4.9: Single-cycle data received buffer: /libraries/position_sensing/bissc/pm_bissc_include.h

```

//
→ *****
//
// This array is loaded with the received single-cycle-data from
// the encoder during the SPI interrupt routine.
//
//
→ *****
extern volatile uint32_t scdRxData[PM_BISSC_SPI_FIFO_MAX_LEVEL];

//

```

4.4 Encoder Interface API

This section describes the Encoder Interface Application Programming Interface (API). Usage of these functions are demonstrated by the system example.

4.4.1 Initialization API

The following functions initialize the system.

void **PM_bissc_generateCRCTable** (uint16_t nBits, uint16_t polynomial, uint16_t *pTable)
Initialize the CRC Lookup Table.

Generate a look-up table of 256 entries for a given CRC polynomial with specified number of bits.

Parameters

- **nBits** – number of bits of the given polynomial
- **polynomial** – polynomial used for the CRC calculations
- **pTable** – pointer to the CRC table

Returns None.

void **PM_bissc_initParams** (PM_bissc_scdStruct_Handle scdParams, PM_bissc_cdStruct_Handle cdParams)
Initialize parameters used to extract position and CRC.

This function initializes mask and shift values used to extract information from the single-cycle data and control data.

This function also initializes the CRC lookup tables.

See also:

PM_bissc_receivePosition

See also:

PM_bissc_doCDTasks

Note: The following parameters must be initialized before calling this the function:

- **crcBits:** Number of CRC bits defined by the utilized polynomial.
 - **positionMTBits:** Number of multi-turn bits supported by the encoder.
 - **positionSTBits:** Number of single-turn bits supported by the encoder.
-

Parameters

- **scdParams** – Single-cycle data Parameters initialized by the function
- **cdParams** – Control data Parameters initialized by the function

Returns none

void **PM_bissc_setFreq** (uint32_t freqDiv)

Scale to generate the MA clock.

This function configures the CLB counters to generate the specified MA clock and the SPI clock frequency.

Parameters

- **freqDiv** – Specified as BISSC_FREQ_DIVIDER in the example's *bissc.h*

Returns none

void **PM_bissc_setupPeriph** (void)

Initialize the CLB tiles.

Initialize the logic of both CLB tiles. The logic configuration code is generated by the library's *.syscfg* file.

Returns none

4.4.2 Run-time API

An example of the runtime flow is shown [the Library Integration Section](#). The following functions are called during run-time.

void **PM_bissc_doCDTasks** (PM_bissc_cdStruct_Handle cdParams, PM_bissc_registerStruct_Handle registerParams) __attribute__((ramfunc))

Command data state-machine.

This state-machine processes the command data frame for read and write of the encoder's registers. When command-data (CDM) is being sent, the this function should be called by the CPU once every BiSS frame.

During system initialization, the CPU sets the state-machine status to **PM_BISSC_CD_NONE**.

To initiate a read or write, the CPU writes the request to `bisscRegisterParams.address` and `bisscRegisterParams.accessType` and then sets the state machine status to `PM_BISSC_CD_NEW_REQUEST`

See also:

PM_bissc_initParams

Attention This example only implements single register read and write control frames (CTS = 1). The following features are not implemented in this release:

- Command control frames where the CTS bit is 0
- Wait states for read/write of registers. This case occurs when the encoder requires more time for a read or write access and holds the start bit low to extend the frame. When this is detected, the state machine will return `PM_BISSC_READ_WRITE_UNSUPPORTED`

Parameters

- **cdParams** – Control data frame parameters
- **registerParams** – Register address, `accessType` and data

Returns

- `cdParams` Updated with the status and `cdm/cds` streams
- `registerParams` Updated `rxData`

void **PM_bissc_haltOperation** (void) __attribute__((ramfunc))

Halt the BiSS-C Subsystem.

This function clears the “START_OPERATION” signal into both tiles

Returns none

uint16_t **PM_bissc_receivePosition** (PM_bissc_scdStruct_Handle scdParams, PM_bissc_cdStruct_Handle cdParams, PM_bissc_encoderStruct_Handle encoderInfo) __attribute__((ramfunc))

Parse the SCD frame.

This function extracts the following information from the single-cycle-data frame:

- CDS: Control Data Slave bit
- MT Position: Multi-Turn data
- ST Position: Single-Turn data
- Error
- Warning
- CRC: Cyclic Redundancy Check for position:error:warning

Returns

- `BISSC_PASS` The received CRC matches the calculated CRC
- `BISSC_FAIL` The received CRC does not match the calculated CRC
- `encoderInfo` is updated with the encoder information

void **PM_bissc_setupSCDTransaction** (PM_bissc_scdStruct_Handle scdParams) __attribute__((ramfunc))
Setup a New BiSS-C Transaction.

This function sets up a new single-cycle transaction.

See also:

This function is called sometime before the *PM_bissc_startOperation* function.

Returns none

void **PM_bissc_startOperation** (PM_bissc_scdStruct_Handle scdParams, uint32_t Freq_us)
__attribute__((ramfunc))

Start the BiSS-C Transaction.

This function starts the transmission of the MA clock.

See also:

This function is called sometime after the single-cycle-data setup function: *PM_bissc_setupSCDTransaction*

Returns none

4.5 Library Private Functions

The following functions are not called directly by the application. They are called from other functions within the library.

void **bissc_generateCRCTable** (uint16_t nBits, uint16_t polynomial, uint16_t *pTable)
Generates CRC tables for a specified polynomial

This function generates table of 256 entries for a given CRC polynomial with specified number of bits (nBits)
Generated tables are stored at the address specified by pTable.

Parameters

- **nBits** – - Number of bits of the given polynomial
- **polynomial** – - Polynomial used for CRC calculations
- **pTable** – - Pointer to the CRC lookup table

Returns None.

static void **bissc_configureCLB** (uint16_t spiClocks, uint16_t dataClocks) __attribute__((ramfunc))
Configure the CLB for a Transaction.

Configures the CLB counters and R0 register in order to generate the required number of MA clocks and SPI clocks for the transaction.

Parameters

- **spiClocks** – The number of SPI clocks required to receive the single-cycle-data into the RX FIFO. This includes any additional clock cycles needed to trigger the SPI RX interrupt.
- **dataClocks** – The number of MA clocks required to receive the single-cycle-data.

Returns None.

static void **bissc_configureSPILength** (uint16_t widthFIFO, uint16_t depthFIFO) __attribute__((ramfunc))
Configure the SPI FIFO depth.

Configures the SPI RX FIFO width and depth as required for the transaction.

Note: The SPI TX FIFO is not used by BiSS-C

Parameters

- **widthFIFO** – is the word length
- **depthFIFO** – is the RX FIFO interrupt level

Returns None.

static uint64_t **bissc_getBits** (uint16_t numBitsToFetch, uint16_t numBitsParsed, uint16_t bufferWidth)
__attribute__((ramfunc))
Extract bits from the received data.

```

///!
///! In the following examples:
///!   - Bits to be fetched are marked "A"
///!   - Bits already parsed are marked as "P"
///!   - "P*" intersection of baseElement with numBitsParsed marks the
///!     last bit previously parsed and the start of data bits to extract.
///!
///! Case 1:
///! The bits to fetch (A) are contained across elements.
///!
///! |<----- width = n+1 ----->|
///! n n-1 n-2 ... 4 3 2 1 0 <-- bit in element
///!
///! x x x ... x x x x x
///! A A A ... A x x x x RxData[baseElement+numElements-1]
///! A A A ... A A A A A RxData[baseElement+numElements-2]
///! ....
///! A A A ... A A A A A RxData[baseElement+1]
///! P P* A ... A A A A A RxData[baseElement]
///!   ^---numBitsParsed
///! P P P ... P P P P P
///! ....
///! P P P ... P P P P P RxData[0]
///!
///! Case 2:
///! The bits to fetch (A) are contained within one element.
///!
///! |<----- width = n+1 ----->|
///! n n-1 n-2 ... 4 3 2 1 0 <-- bit in element
///!
///! x x x ... x x x x x
///! P P* A ... A A A A x RxData[baseElement]

```

(continues on next page)

(continued from previous page)

```

//!      ^    <--bitsToFetch -->|
//!      |
//!      -----numBitsParsed
//!
//!

```

Attention This function makes the following assumptions:

- Data is right justified in the global array named `scdRxData`.
- Data is ordered such that element 0 is the most-significant word
- Fetching bits beyond the size of the array is not checked

Parameters

- **numBitsToFetch** – Number of bits to extract and return.
- **numBitsParsed** – How many bits were previously parsed. Corresponds to the starting place for bit extraction.
- **bufferWidth** – Word size of the SPI FIFO that captured the data.

Returns Bits extracted from the `scdRxData` array.

```
uint16_t bissc_getCRC (uint16_t crcStart, uint16_t nBitsPoly, uint16_t *msg, uint16_t *crcTable, uint16_t
                      msgBytes) __attribute__((ramfunc))
```

Calculate a CRC using a lookup table.

Parameters

- **crcStart** – Initial value. This is typically 0.
- **nBitsPoly** – The number of bits in the polynomial
- **msg** – A pointer to the data to calculate the CRC for
- **crcTable** – A pointer to the CRC lookup table
- **msgBytes** – The number of 8-bit bytes in the message. For example: If the message is 16-bits, then `msgBytes` is 2. If the message uses a partial byte then round up. For example: 18-bits then `msgBytes` = 3.

Returns The calculated CRC value.

```
static void bissc_resetCLB (void) __attribute__((ramfunc))
```

Reset the CLB.

Reset portions of the CLB after a transaction or to recover from a failed transaction.

Returns None.

```
static void bissc_setCDMBit (enum cdmValue value) __attribute__((ramfunc))
```

Set the CDM bit for the next SCD transaction.

```

//

enum cdmValue
{

```

(continues on next page)

(continued from previous page)

```

PM_BISSC_SCD_CDM_ONE = 0U,      // CDM = 1
PM_BISSC_SCD_CDM_ZERO = 1U,     // CDM = 0
PM_BISSC_SCD_CDM_DEFAULT = 1U,  // CDM default = 0
};

//

```

The value is sent to the CLB tiles through the CLB's GPREG.

Parameters

- **value** – to be output during the BiSS timeout. The inverse of value is interpreted to be the CDM bit by the encoder.

Returns None.

4.6 Protocol Defines

The library private header file includes a number of BiSS-C protocol-based defines. Some of these are shown below for reference. Information on specific fields and commands can be found in the [glossary](#).

4.6.1 Command Data (CD) Frame

Listing 4.10: /libraries/position_sensing/bissc/include/pm_bissc_internal_include.h

```

//
//
→ *****
//
// Length, in bits, of various portions of the control data master_
→ (CDM) frame
// _S is the shift required to right justify the field
// _M indicates the mask to extract the bits
//
//
// CDM:
// START1 : CTS : ID : ADDR : CRC1 R : W : START2 : Data : CRC2 :
→ STOP
// 1 1 3 7 4 1 1 1 8 4
→ 1 bits
// <----- Read -----><---- 12 0s + STOP
→ --->
// <----- Write -----
→ --->
// <----- 32 bits -----
→ --->
//
//
→ *****
#define PM_BISSC_CD_BITS 32U
#define PM_BISSC_CD_START1_BITS 1U

```

(continues on next page)

(continued from previous page)

```

#define PM_BISSC_CD_START1_S          31U
#define PM_BISSC_CD_START1_M          1U
#define PM_BISSC_CD_CTS_BITS          1U
#define PM_BISSC_CD_CTS_S             30U
#define PM_BISSC_CD_CTS_M             1U
#define PM_BISSC_CD_ID_BITS           3U
#define PM_BISSC_CD_ID_S              27U
#define PM_BISSC_CD_ID_M              0x0007U
#define PM_BISSC_CD_ADDR_BITS         7U
#define PM_BISSC_CD_ADDR_S            20U
#define PM_BISSC_CD_ADDR_M            0x007FU
#define PM_BISSC_CD_ID_ADDR_BITS      10U
#define PM_BISSC_CD_CTS_ID_ADDR_S     20U
#define PM_BISSC_CD_CTS_ID_ADDR_M     0x07FFU
#define PM_BISSC_CD_ADDR_CRC_BITS     4U
#define PM_BISSC_CD_ADDR_CRC_S        16U
#define PM_BISSC_CD_ADDR_CRC_M        0x000FU
#define PM_BISSC_CD_RW_BITS           2U
#define PM_BISSC_CD_RW_S              14U
#define PM_BISSC_CD_RW_M              0x003U
#define PM_BISSC_CD_START2_BITS       1U
#define PM_BISSC_CD_START2_S          13U
#define PM_BISSC_CD_START2_M          0x0001U
#define PM_BISSC_CD_DATA_BITS         8U
#define PM_BISSC_CD_CRC2_BITS         4U
#define PM_BISSC_CD_CRC2_S            1U
#define PM_BISSC_CD_CRC2_M            0x000FU
#define PM_BISSC_CD_RWS_S             13U
#define PM_BISSC_CD_RWS_M             0x0007U
#define PM_BISSC_CD_DATA_BITS         8U
#define PM_BISSC_CD_DATA_S            5U
#define PM_BISSC_CD_DATA_M            0x00FFU
#define PM_BISSC_CD_DATA_CRC_BITS     4U
#define PM_BISSC_CD_DATA_CRC_S        1U
#define PM_BISSC_CD_DATA_CRC_M        0x000FU
#define PM_BISSC_CD_STOP_BITS         1U
#define PM_BISSC_CD_STOP_S            0U
#define PM_BISSC_CD_STOP_M            0x0001U

//
→ *****
//
// Defines to set or check if a bit is set (high) or cleared (low)
//
//
→ *****
#define PM_BISSC_CD_START_SET          1UL
#define PM_BISSC_CD_START_CLEAR        0UL
#define PM_BISSC_CD_CTS_SET           1UL
#define PM_BISSC_CD_CTS_CLEAR          0UL
#define PM_BISSC_CD_STOP_SET           1UL

```

(continues on next page)

(continued from previous page)

```

//
→ *****
//
// Defines to compare bits against
//
//
→ *****
#define PM_BISSC_CD_ID                0U
#define PM_BISSC_CD_READ              0b10U    // R+W for read
#define PM_BISSC_CD_WRITE             0b01U    // R+W for write
#define PM_BISSC_CD_RW_INVALID        0b11U    // Both R & W high
→ is not valid
#define PM_BISSC_CD_READ_START        0b101U   // R+W+S for read
#define PM_BISSC_CD_WRITE_START       0b011U   // R+W+S for write
#define PM_BISSC_CD_RW_UNSUPPORTED    0b110U

//

```

4.6.2 Single Cycle Data (SCD) Frame

Listing 4.11: /libraries/position_sensing/bissc/include/pm_bissc_internal_include.h

```

//
//
→ *****
//
// _S is the shift required to right justify the field
// _M indicates the mask to extract the bits
//
// The number of multi-turn (MT) and single-turn (ST) position bits
// is often read out of the encoder on start-up. For this reason,
→ the MT shift
// along with the MT and ST mask values are calculated at run-time.
//
// Shift and mask values to extract portions of the scd_raw data
//     ST_POSITION : MT_POSITION : error : warning
//     POSITION_BITS      1      1      bits
//
//
→ *****
#define PM_BISSC_SCD_RAW_ERROR_S      0U
#define PM_BISSC_SCD_RAW_ERROR_M      1U
#define PM_BISSC_SCD_RAW_WARNING_S    1U
#define PM_BISSC_SCD_RAW_WARNING_M    1U
#define PM_BISSC_SCD_RAW_POSITION_S   2U
#define PM_BISSC_SCD_RAW_ST_POSITION_S 2U

//
→ *****
//

```

(continues on next page)

(continued from previous page)

```

// Number of bits for each field within a single-cycle-data frame
//
//
→ *****
#define PM_BISSC_SCD_ERROR_BITS          1U
#define PM_BISSC_SCD_WARNING_BITS        1U
#define PM_BISSC_SCD_START_BITS          1U
#define PM_BISSC_SCD_CDS_BITS            1U

//
→ *****
//
// Defines to set or check if a bit is set (high) or cleared (low)
// Note: CDM is transmitted inverted
//
// The CDM is controlled by the CPU through the GPREG input
//
//
→ *****
//
//! [BISSC_ENUM_CDMVALUE_SNIPPET]
//

enum cdmValue
{
    PM_BISSC_SCD_CDM_ONE = 0U,          // CDM = 1
    PM_BISSC_SCD_CDM_ZERO = 1U,         // CDM = 0
    PM_BISSC_SCD_CDM_DEFAULT = 1U,     // CDM default = 0
};

//
//! [BISSC_ENUM_CDMVALUE_SNIPPET]
//

//
// GPREG bit definitions
//
#define PM_BISSC_START_OPERATION_BIT      0U
#define PM_BISSC_CDM_CONTROL_BIT         7U

//

```

ADDITIONAL RESOURCES

5.1 BiSS-C License

BiSS is a trademark of iC-Haus GmbH. Developers must obtain their own BiSS license. The purchase of TI MCUs does not automatically include a license to use the BiSS IPs (e.g. for marketing). Users of the BiSS Interface have to apply for their own license [Information can be found at **biss-interface.com**](http://biss-interface.com).

5.2 BiSS-C Protocol Description

For more details on the protocol and implementation aspects, see the [BiSS-C Interface](http://biss-interface.com) website.

5.3 Abbreviations and Acronyms

API Application Programming Interface. The definition of the library which enables the encoder interface protocol.

BiSS In this document, BiSS Interface or BiSS refer to the BiSS-C protocol introduced in 2007. BiSS-C stands for Bidirectional Serial Synchronous Continuous mode. [Information can be found at **biss-interface.com**](http://biss-interface.com). For more details on the protocol and implementation aspects, see the [BiSS-C Interface](http://biss-interface.com) website.

BiSS frame Single-cycle-data plus one bit of control-data. The BiSS frame is sent as a response every BiSS cycle.

CD From the BiSS-C specification. Control Data. Data transmitted one bit per BiSS cycle. For example, the CD permits the reading and writing of encoder registers.

CDM From the BiSS-C specification. Control Data Master. One bit of control data, CD, sent each BiSS frame by the controller to the encoder.

CDS From the BiSS-C specification. Control Data Slave. One bit of control data, CD, sent each BiSS frame by the encoder to the controller.

CLB Configurable Logic Block peripheral on C2000 devices.

CRC Cyclic Redundancy Check.

CTS From the BiSS-C specification. ConTrol Select bit. A BiSS-C control data (CD) frame begins with a start bit followed by the CTS bit. If CTS is 0, the control frame is a command frame. If CTS is 1, then the control frame is a register access frame.

MA From the BiSS-C specification. Master clock. This pulse train output is sent from the controller to the encoder/sensor. It consists of the encoder clock plus one bit of control data from the master.

PM Position Manager. Foundation hardware and software on C28x devices for position sensing

PM_bissc or PM_BISSC Prefix used for the library functions and #defines

Point-to-point Topology A configuration in which only one device (encoder or sensor) is interfaced to the master.

SCD Single-Cycle Data. Information transmitted in one BiSS frame. This includes the position information.

SL From the BiSS-C specification. Slave return. Data input to the BiSS-C master from the slave. This is the encoder response including position and one bit of CD data each BiSS frame.

SPI Serial Peripheral Interface

CHANGELOG

6.1 C2000 MotorControl SDK Version 5.03.00.00

- Initial release for the F28P65x

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, enhancements, improvements and other changes to its semiconductor products and services per JESD46, latest issue, and to discontinue any product or service per JESD48, latest issue. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All semiconductor products (also referred to herein as “components”) are sold subject to TI’s terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its components to the specifications applicable at the time of sale, in accordance with the warranty in TI’s terms and conditions of sale of semiconductor products. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by applicable law, testing of all parameters of each component is not necessarily performed.

TI assumes no liability for applications assistance or the design of Buyers’ products. Buyers are responsible for their products and applications using TI components. To minimize the risks associated with Buyers’ products and applications, Buyers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right relating to any combination, machine, or process in which TI components or services are used. Information published by TI regarding third-party products or services does not constitute a license to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of significant portions of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI components or services with statements different from or beyond the parameters stated by TI for that component or service voids all express and any implied warranties for the associated TI component or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

Buyer acknowledges and agrees that it is solely responsible for compliance with all legal, regulatory and safety-related requirements concerning its products, and any use of TI components in its applications, notwithstanding any applications-related information or support that may be provided by TI. Buyer represents and agrees that it has all the necessary expertise to create and implement safeguards which anticipate dangerous consequences of failures, monitor failures and their consequences, lessen the likelihood of failures that might cause harm and take appropriate remedial actions. Buyer will fully indemnify TI and its representatives against any damages arising out of the use of any TI components in safety-critical applications.

In some cases, TI components may be promoted specifically to facilitate safety-related applications. With such components, TI’s goal is to help enable customers to design and create their own end-product solutions that meet applicable functional safety standards and requirements. Nonetheless, such components are subject to these terms.

No TI components are authorized for use in FDA Class III (or similar life-critical medical equipment) unless authorized officers of the parties have executed a special agreement specifically governing such use.

Only those TI components which TI has specifically designated as military grade or “enhanced plastic” are designed and intended for use in military/aerospace applications or environments. Buyer acknowledges and agrees that any military or aerospace use of TI components which have not been so designated is solely at the Buyer’s risk, and that Buyer is solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI has specifically designated certain components which meet ISO/TS16949 requirements, mainly for automotive use. Components which have not been so designated are neither designed nor intended for automotive use; and TI will not be responsible for any failure of such components to meet such requirements.

DISCLAIMER

ALL CONTENT AND MATERIALS ON THIS SITE ARE PROVIDED “AS IS”. TI AND ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THESE MATERIALS FOR ANY PURPOSE AND DISCLAIM ALL WARRANTIES AND CONDITIONS WITH REGARD TO THESE MATERIALS, INCLUDING BUT NOT LIMITED TO, ALL IMPLIED WARRANTIES AND CONDITIONS OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT OF ANY THIRD PARTY INTELLECTUAL PROPERTY RIGHT. NO LICENSE, EITHER EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, IS GRANTED BY TI. USE OF THE INFORMATION ON THIS SITE MAY REQUIRE A LICENSE FROM A THIRD PARTY, OR A LICENSE FROM TI.

Content on this site may contain or be subject to specific guidelines or limitations on use. All postings and use of the content on this site are subject to the Terms of Use of the site; third parties using this content agree to abide by any limitations or guidelines and to comply with the Terms of Use of this site. TI and its suppliers reserve the right to make corrections, deletions, modifications, enhancements, improvements and other changes to the content and materials, its products, programs and services at any time or to move or discontinue any content, products, programs, or services without notice.

INDEX

B

bissc_configureCLB (*C function*), 33
bissc_configureSPILength (*C function*), 34
bissc_generateCRCTable (*C function*), 33
bissc_getBits (*C function*), 34
bissc_getCRC (*C function*), 35
bissc_init (*C function*), 19
bissc_resetCLB (*C function*), 35
bissc_runSystem (*C function*), 19
bissc_setCDMBit (*C function*), 35
bissc_setupCDTest (*C function*), 20
bissc_spiRxISR (*C function*), 20

P

PM_bissc_doCDTasks (*C function*), 31
PM_bissc_generateCRCTable (*C function*), 30
PM_bissc_haltOperation (*C function*), 32
PM_bissc_initParams (*C function*), 30
PM_bissc_receivePosition (*C function*), 32
PM_bissc_setFreq (*C function*), 31
PM_bissc_setupPeriph (*C function*), 31
PM_bissc_setupSCDTransaction (*C function*),
32
PM_bissc_startOperation (*C function*), 33